// GUIDA GRATIS · 5 PLUGIN PER CLAUDE CODE

I 5 PLUGIN DEL REEL, *installati passo passo.*

Per ognuno: cosa fa davvero, i comandi esatti per installarlo in Claude Code, cosa devi vedere quando funziona, gli errori comuni e i **limiti onesti**. Tre li abbiamo provati noi l'11/11/2026 e trovi l'output vero; per gli altri due lo diciamo chiaramente.

00 · La base	come si installa un plugin in Claude Code	pag. 02
01 · OmniRoute	un solo indirizzo per centinaia di provider AI	pag. 03
02 · Superpowers	brainstorming, piano, test e review prima del codice	pag. 04
03 · Caveman	risposte più corte, codice identico	pag. 05
04 · Claude Code Setup	ti dice quali automazioni servono al tuo progetto	pag. 06
05 · Claude Security	un team di agenti cerca le vulnerabilità	pag. 07
Verifica e link	il test di ognuno, cosa costa, i link ufficiali	pag. 08-09

// COSA TI SERVE

01 Claude Code installato e un piano Claude
I plugin girano dentro Claude Code e consumano il tuo uso di Claude (Pro, Max, Team o API). Installazione a pag. 02.

02 Node.js e npm
Per OmniRoute (`npm install -g`) e per la CLI di Caveman (Node 22.13 o più recente). Il solo plugin Caveman non li richiede.

03 Python 3.9+ e un repo git
Li chiede Claude Security: Python sul PATH, git per scansionare le modifiche e proporre patch.

// COME USARLA

Non serve installarli tutti. Parti da **uno** che risolve un problema che hai oggi (token che finiscono, codice scritto di fretta, sicurezza), fai i passi in ordine e il test di pag. 08. I comandi sono copiati dai README ufficiali, letti l'11/11/2026: se nel frattempo sono cambiati, vale il README del link.

// PRIMA DI INIZIARE · LA BASE

00 COME SI INSTALLA *un plugin.*

Un plugin è un pacchetto di skill, comandi, agenti e hook che si aggiunge a Claude Code. Arriva da un **marketplace**: quello ufficiale di Anthropic (`claude-plugins-official`) Claude Code lo aggiunge da solo al primo avvio nel terminale; gli altri li aggiungi tu con un comando.

```
// INSTALLA CLAUDE CODE (quickstart ufficiale)
curl -fsSL https://claude.ai/install.sh | bash     # macOS, Linux, WSL
irm https://claude.ai/install.ps1 | iex             # Windows PowerShell
claude --version                                    # deve stampare la versione
cd /cartella/del/progetto && claude                 # al primo avvio accedi dal browser
```

01 **Plugin dal marketplace ufficiale**
Dentro Claude Code scrivi `/plugin install nome@claude-plugins-official`. Si apre il pannello del plugin: lo leggi e scegli lo scope.

02 **Plugin da un altro marketplace**
Prima `/plugin marketplace add autore/repo`, poi `/plugin install nome@marketplace`. Vale per Caveman.

03 **Scegli lo scope**
User: in tutti i tuoi progetti (`~/.claude/settings.json`). **Project**: per tutti quelli che lavorano nel repo. **Local**: solo tu, solo in quel repo.

04 **Ricarica**
Se il pannello lo chiede, `/reload-plugins`. Poi controlli nella scheda Installed di `/plugin` o con `claude plugin list` dal terminale.

```
// COSA DEVI VEDERE · OUTPUT VERO DI claude plugin list (11/11/2026)
Installed plugins:
  ❯ superpowers@claude-plugins-official
    Version: 7.0.0
    Scope: user
    Status: ✔ enabled
```

Errori comuni — installare da un marketplace non aggiunto («marketplace not found»): aggiungilo prima. Installare in scope Project un plugin che vuoi solo tu: lo attivi a tutto il team.

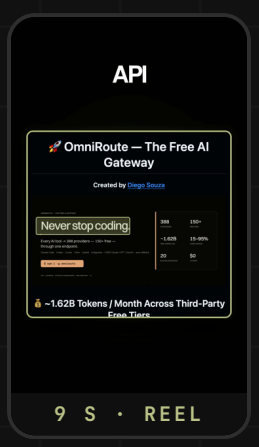
Consiglio — per toglierne uno: `claude plugin disable nome` lo spegne, `claude plugin uninstall nome` lo rimuove. Installa un plugin alla volta: se qualcosa cambia, sai chi è stato.

FONTI · code.claude.com/docs/en/quickstart · code.claude.com/docs/en/discover-plugins

// PLUGIN 01 · OMNIROUTE

01 CENTINAIA DI PROVIDER *dietro un indirizzo.*

Cosa fa: un **gateway AI** open source (MIT, di Diego Souza) che gira sul tuo computer. Tu colleghi i provider con i **tuoi account**, lui espone un solo indirizzo e, con il modello `auto`, passa al provider successivo quando uno finisce i limiti. Non è un plugin del marketplace: è un programma a parte.



01 Installa e avvia

`npm install -g omniroute`, poi `omniroute`. Dashboard su `localhost:20128`.

02 Collega un provider

Dashboard › Providers: scegli un provider, crea l'account da lui, incolla la sua chiave. Leggi prima i suoi termini e i limiti gratuiti.

03 Fai partire Claude Code attraverso OmniRoute

`omniroute run claude --model auto`: non scrive file di configurazione, inietta le variabili solo per quel processo.

// VERIFICA E CONFIGURAZIONE FISSA (README e doc CLI-TOOLS)

```
curl http://localhost:20128/v1/models -H "Authorization: Bearer LA_TUA_CHIAVE"
# deve elencare i modelli dei provider collegati
# in alternativa, fisso in ~/.claude/settings.json (senza /v1):
"env": { "ANTHROPIC_BASE_URL": "http://localhost:20128",
         "ANTHROPIC_AUTH_TOKEN": "LA_TUA_CHIAVE_OMNIROUTE" }
```

⚡ Zero-config — just use `auto`

No combo to create. Set your model to `auto` (or a variant) and OmniRoute builds a virtual combo from your connected providers, scored live:

Model ID	What it optimizes for
`auto`	🎯 Balanced default (LKGP — sticks to your last good provider)
`auto/coding`	👨‍💻 Quality-first weights for code generation
`auto/fast`	⚡ Lowest latency first
`auto/cheap`	💰 Lowest provider cost first

GITHUB · OMNIROUTE · AUTO

// LIMITI ONESTI · LEGGILI PRIMA

Il «~1,62 miliardi di token al mese» è, dal README, una **stima sommata dei piani gratuiti di provider terzi**, non token regalati da OmniRoute: ognuno va attivato con il tuo account e decide lui limiti e termini. Con `auto` Claude Code risponde con **i modelli di quei provider**, non con Claude, a meno che non colleghi un tuo account Anthropic. I prompt e il codice vanno al provider scelto per ogni richiesta.

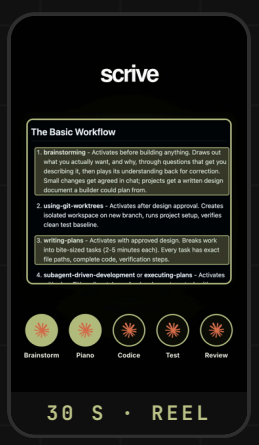
Errori comuni — la doc mostra `cat > ~/.claude/settings.json`: sovrascrive il file con le tue impostazioni, aggiungi solo il blocco `env`. Un'installazione nuova può non avere provider utilizzabili da `auto`. **Non l'abbiamo eseguito noi** (servono account presso i provider): prova su un progetto di test.

FONTI · github.com/cheaper-inference/OmniRoute · github.com/cheaper-inference/OmniRoute/blob/main/docs/reference/CLI-TOOLS.md

// PLUGIN 02 · SUPERPOWERS

02 PRIMA IL PIANO, *poi il codice.*

Cosa fa: un insieme di skill (MIT, di Jesse Vincent, `obra/superpowers`) che cambia il modo di lavorare di Claude: prima di scrivere fa domande e un design, poi un piano a passi piccoli, poi sviluppo con test (red/green) e una code review. È nel marketplace ufficiale.



30 S · REEL

01 Installa
In Claude Code: `/plugin install superpowers@claude-plugins-official`, scope User.

02 Oppure dal suo marketplace
`/plugin marketplace add obra/superpowers-marketplace` e `/plugin install superpowers@superpowers-marketplace`.

03 Apri una sessione nuova
Le skill partono da sole quando servono: non c'è un comando da lanciare.

The Basic Workflow

1. **brainstorming** - Activates before building anything. Draws out what you actually want, and why, through questions that get you describing it, then plays its understanding back for correction. Small changes get agreed in chat; projects get a written design document a builder could plan from.
2. **using-git-worktrees** - Activates after design approval. Creates isolated workspace on new branch, runs project setup, verifies clean test baseline.
3. **writing-plans** - Activates with approved design. Breaks work into bite-sized tasks (2-5 minutes each). Every task has exact file paths, complete code, verification steps.
4. **subagent-driven-developm**er or executing-plans - Activates with plan. Either dispatches a fresh subagent per task with t

GITHUB · SUPERPOWERS

// ESEMPIO VERO · LA PROVA DEL README, ESEGUITA DA NOI L'11/11/2026

Il README dice: sessione nuova, scrivi `Let's make a react todo list`: se è installato bene parte **brainstorming** prima di qualsiasi codice. Noi l'abbiamo lanciata in modalità non interattiva (`claude -p`, senza nessuno che risponde): lì il brainstorming **non è partito** e Claude ha scritto direttamente il codice. Quindi la prova va fatta in una sessione normale, dove puoi rispondere alle domande.

Errori comuni — aspettarsi che lavori uguale in `claude -p` o negli script: è pensato per la conversazione. Usarlo per una modifica di una riga: il flusso completo (piano, worktree, review) costa più tempo e token.

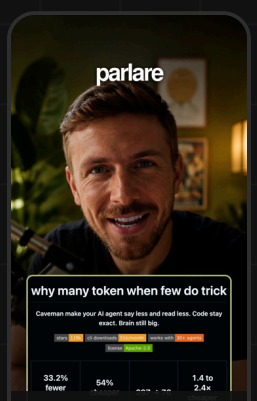
Consiglio — quando ti propone il design, leggilo davvero e correggilo: è il punto in cui risparmi più ore. Un piano sbagliato approvato in fretta produce codice sbagliato ben testato.

FONTI · github.com/obra/superpowers · claude.com/plugins/superpowers

// PLUGIN 03 · CAVEMAN

03 MENO PAROLE, *stesso codice.*

Cosa fa: Claude risponde «da cavernicolo»: risposta prima, frasi brevi, niente giri di parole, ma codice, comandi e messaggi di errore restano identici. Di Julius Brussee (`JuliusBrussee/caveman` , Apache-2.0). Dalla versione 4 ha anche parti che riducono quello che Claude **legge** (log, output dei test).



36 S · REEL

01 Installa il plugin (dentro Claude Code)
`/plugin marketplace add JuliusBrussee/caveman` poi `/plugin install caveman@caveman` .

02 Oppure la CLI completa
`npx @caveman-ai/cli@latest` (Node 22.13+): trova gli agenti sul computer, ti mostra cosa cambia, premi Invio.

03 Usalo
Parte da solo nelle sessioni interattive. Comandi: `/caveman` , `/ultracave` (più corto), `/caveman off` .

// ESEMPIO VERO · STESSA DOMANDA, ESEGUITA DA NOI L'11/11/2026

```
Domanda: «Perché il mio componente React si ri-renderizza se gli passo
un oggetto inline come prop?»
  senza Caveman      916 token di risposta
  con /caveman       580 token di risposta   (-37%)
Risposta con /caveman (inizio, vera):
  «Ogni render del padre crea un oggetto nuovo. … React confronta le props
   per riferimento (Object.is), non per contenuto.» + stesso fix useMemo
```

Normal agent · 63 tokens	Caveman agent · 20 tokens
The reason your React component is re-rendering is likely because you're creating	New object ref each render, so React re-renders. Wrap the prop in

GITHUB · CAVEMAN README · 11/11

Errori comuni — pensare che dimezzi il conto: nei numeri del README stesso, rispetto a un semplice «rispondi conciso» `/caveman` toglie il 3% in mediana e `/ultracave` il 35%. In `claude -p` non parte da solo: nel nostro test senza `/caveman` il risparmio era solo del 9%.

Consiglio — la CLI manda statistiche d'uso (mai prompt o codice): si spengono con `caveman telemetry off` . Per spiegazioni lunghe da leggere con calma, spegnilo con `/caveman off` .

FONTI · github.com/JuliusBrussee/caveman · github.com/JuliusBrussee/caveman/blob/main/INSTALL.md

// PLUGIN 04 · CLAUDE CODE SETUP

04 QUALI AUTOMAZIONI *servono a te.*

Cosa fa: plugin **ufficiale Anthropic**. Legge il tuo progetto e ti consiglia le 1-2 automazioni migliori per categoria: server MCP, skill, hook, subagenti, slash command. Dal README: è **in sola lettura**, analizza ma non modifica file. Le automazioni le crei tu, se ti convincono.



47 S · REEL

01 Installa
`/plugin install claude-code-setup@claude-plugins-official`, poi `/reload-plugins` se richiesto.

02 Apri Claude Code nella cartella del progetto
Più il progetto è reale (codice, README, script), più i consigli sono precisi.

03 Chiedilo a parole
`recommend automations for this project`, oppure «help me set up Claude Code», «what hooks should I use?». Puoi aggiungere «rispondi in italiano».

// ESEMPIO VERO · ESEGUITO DA NOI L'11/11/2026 SU UN NOSTRO PROGETTO PYTHON (estratto)

Cosa fa il progetto: pipeline Python che genera un PDF da HTML (Playwright).
⚡ Hook 1 · PostToolUse su Edit|Write: dopo ogni modifica a build.py o base.css rilancia la build e il controllo di impaginazione.
⚡ Hook 2 · PreToolUse: blocca le modifiche ai file generati (guida.html) con «file generato: modifica build.py».
🤖 Subagenti · verifica-fonti (riapre i link citati), revisore-visivo.
🎯 Skill · uno scaffold per creare nuove guide dal motore esistente.
🔌 MCP · «Non serve aggiungere niente.»



Errori comuni — aspettarsi che installi qualcosa: ti dà consigli e lo snippet, l'hook lo scrivi tu in `.claude/settings.json`. Lanciarlo in una cartella vuota: senza codice non ha niente da consigliare.

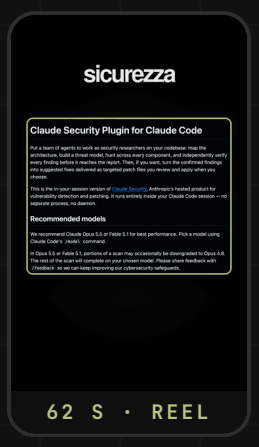
Consiglio — prendi un consiglio alla volta, il più semplice (di solito un hook), e provalo per una settimana prima di aggiungerne altri.

FONTI · github.com/anthropics/claude-plugins-official/tree/main/plugins/claude-code-setup

// PLUGIN 05 · CLAUDE SECURITY

05 LE VULNERABILITÀ, *prima degli altri.*

Cosa fa: plugin **ufficiale Anthropic**, versione «dentro la sessione» del prodotto Claude Security. Un team di agenti mappa l'architettura, costruisce un modello delle minacce, cerca falle in ogni componente e fa **verificare ogni risultato** da agenti indipendenti prima di metterlo nel report. Su richiesta propone patch.



62 S · REEL

01 Installa e ricarica
`/plugin install claude-security@claude-plugins-official` e `/reload-plugins`. Se dice «marketplace not found»: `/plugin marketplace add anthropics/claude-plugins-official`.

02 Apri il menu
`/claude-security`: Scan codebase (tutto o una parte), Scan changes (branch, PR, commit), Suggest patches.

03 Oppure scrivilo diretto
`/claude-security scan my branch at high`. Prima di partire ti chiede di confermare tempo e costo in token.

Getting started

Run `/claude-security` for the menu. It offers the three jobs the plugin does:

Job	What it scans
Scan codebase	The whole repository, or a scoped part of it
Scan changes	This b

GITHUB · CLAUDE-SECURITY · GETTING STARTED · 11/11

// COSA DEVI VEDERE (dal README)

Una cartella `CLAUDE-SECURITY-<data>/` nel repo con `CLAUDE-SECURITY-RESULTS.md` (ogni falla con impatto, scenario di attacco, gravità da CRITICAL a LOW, confidenza), lo stesso in JSONL e SARIF per GitHub, e un `.gitignore` suo. Le patch finiscono in `patches/F<n>.patch`: **non applica, non committa e non pubblica niente**; le applichi tu con `git apply`.

Errori comuni — scansionare codice di terzi di cui non ti fidi: il plugin non aggiunge isolamento, Anthropic consiglia di farlo dentro sandbox-runtime. Prendere un report pulito come «nessuna falla»: le scansioni non sono deterministiche e non sostituiscono SAST e review.

Consiglio — non l'abbiamo eseguito noi: una scansione completa usa molti agenti e molti token. Parti da `Scan changes` sul tuo ultimo branch, è la scansione più piccola.

FONTI · github.com/anthropics/claude-plugins-official/tree/main/plugins/claude-security · github.com/anthropic-experimental/sandbox-runtime

// VERIFICA E COSTI

COME SAI CHE *funziona.*

Un plugin è installato bene quando passa una prova vera, non quando compare nella lista. Fai il test della riga, poi usalo una settimana sul tuo lavoro prima di aggiungerne un altro.

01 · OmniRoute	`curl …/v1/models` elenca i modelli; `claude "say hello"` risponde	**2 test**
02 · Superpowers	sessione nuova, «Let's make a react todo list»: domande prima del codice	**1 test**
03 · Caveman	stessa domanda con e senza `/caveman`: risposta più corta, stesso fix	**1 test**
04 · Setup	«recommend automations for this project»: consigli per categoria	**1 test**
05 · Security	`/claude-security` apre il menu; poi Scan changes su un branch	**1 test**

// COSA COSTA (dai README, 11/11/2026)

Tutti e 5	software gratuito e open source o plugin ufficiali gratuiti	**gratis**
Claude Code	consumo del tuo piano Claude o dell'API	**piano Claude**
OmniRoute	i piani gratuiti dei provider, con i tuoi account; oltre i limiti, i loro prezzi	**dipende dal provider**
Caveman · Auto	unica parte che chiede un account Caveman, gratuito	**gratis**
Claude Security	ogni scansione consuma token: ti chiede conferma del costo	**token del tuo piano**

01 **Uno alla volta**
Installane uno, fai il test, usalo. Se qualcosa cambia nel comportamento di Claude sai da dove arriva.

02 **Controlla cosa carichi**
Con `claude plugin details nome` vedi cosa contiene un plugin e quanti token aggiunge al contesto.

> **Consiglio —** Caveman e Superpowers cambiano il modo di rispondere di Claude: se un giorno ti sembra «strano», prova a disattivarli con `claude plugin disable` prima di cercare altrove.

// LINK UFFICIALI

TUTTO QUELLO CHE TI *serve.*

README e documentazioni citate nella guida, aperte e controllate l'11/11/2026.

Claude Code · installazione	code.claude.com/docs/en/quickstart	00
Claude Code · installare plugin	code.claude.com/docs/en/discover-plugins	00
Marketplace ufficiale Anthropic	github.com/anthropics/claude-plugins-official	00 02 04 05
OmniRoute	github.com/cheaper-inference/OmniRoute	01
OmniRoute · configurare Claude Code	github.com/cheaper-inference/OmniRoute/blob/main/docs/reference/CLI-TOOLS.md	01
Superpowers	github.com/obra/superpowers	02
Superpowers · pagina plugin	claude.com/plugins/superpowers	02
Caveman	github.com/JuliusBrussee/caveman	03
Caveman · installazione	github.com/JuliusBrussee/caveman/blob/main/INSTALL.md	03
Claude Code Setup	github.com/anthropics/claude-plugins-official/tree/main/plugins/claude-code-setup	04
Claude Security	github.com/anthropics/claude-plugins-official/tree/main/plugins/claude-security	05
sandbox-runtime	github.com/anthropic-experimental/sandbox-runtime	05

// SE QUALCOSA VA STORTO

claude plugin list	cosa è installato, versione, scope, stato	terminale
claude plugin disable nome	lo spegne senza toglierlo	terminale
claude plugin uninstall nome	lo rimuove	terminale
/reload-plugins	applica le modifiche nella sessione aperta	Claude Code

// RICORDA

Un plugin non rende Claude più intelligente: cambia **come lavora** (Superpowers, Caveman), **cosa ti consiglia** (Setup), **cosa controlla** (Security) o **con quali modelli gira** (OmniRoute). Ognuno aggiunge istruzioni al contesto e, a volte, token. Tieni quelli che superano la prova di pag. 08 sul tuo lavoro vero, togli gli altri.

Versioni, comandi e limiti dei provider cambiano: se qualcosa non corrisponde, vale il README del link.